

Dependências, ILP e otimização de laços em C

Matheus Marinho

21 de agosto de 2026

1. Objetivo

O objetivo desta tarefa foi investigar o paralelismo ao nível de instrução (*Instruction-Level Parallelism*, ILP). Foram implementados um laço de inicialização, uma soma com dependência entre iterações e versões da soma com 2, 4 e 8 acumuladores independentes. Os tempos foram comparados com os níveis de otimização `-O0`, `-O2` e `-O3`.

2. Os três tipos de laço

No laço de **inicialização**, cada iteração calcula e escreve uma posição diferente do vetor. Uma iteração não precisa do resultado da anterior, permitindo que o processador execute várias instruções ao mesmo tempo.

Na **soma dependente**, existe apenas um acumulador:

```
soma ← soma + vetor[i]
```

Cada adição precisa esperar o valor de `soma` produzido pela adição anterior. Essa cadeia cria uma dependência verdadeira. Enquanto o resultado não fica pronto, podem surgir espaços sem trabalho útil, conhecidos como **bolhas no pipeline**.

O terceiro estilo quebra a cadeia usando acumuladores independentes. Com quatro variáveis, por exemplo, `soma0`, `soma1`, `soma2` e `soma3` podem avançar sem esperar umas pelas outras. Ao final, elas são combinadas para formar o resultado.

Foram testados 2, 4 e 8 acumuladores para identificar até que ponto criar cadeias independentes continua trazendo ganho.

3. Metodologia

- Vetor com 16 milhões de elementos do tipo `double`, ocupando aproximadamente 122 MiB.
- Tempo de parede medido com `clock_gettime(CLOCK_MONOTONIC)`.
- Quantidade de repetições dobrada automaticamente até cada medição durar pelo menos 0,6 segundo.
- Três execuções por combinação; a tabela apresenta a mediana.
- Cada teste executado em um novo processo. Assim, a soma dependente e as somas múltiplas não são medidas uma imediatamente depois da outra.
- Compilação com GCC 16.1.0 e as mesmas opções, mudando apenas `-O0`, `-O2` ou `-O3`.
- Resultado observado por uma variável `volatile` e funções marcadas como `noinline`, evitando que o compilador elimine ou junte os testes.

O vetor é inicializado por cada processo antes da soma. Como ele é maior que a cache compartilhada do processador, não permanece inteiro na cache. Isso reduz a possibilidade de um teste ser favorecido pelo acesso realizado no teste anterior.

4. Resultados

Tempos medianos de uma passagem completa pelo vetor, em milissegundos:

Laço	-O0	-O2	-O3	Ganho O0 → O3
Inicialização	22,324	7,242	6,528	3,42×
Soma dependente	48,504	12,334	12,405	3,91×
2 acumuladores	24,843	6,742	6,633	3,75×
4 acumuladores	13,017	4,388	4,390	2,97×
8 acumuladores	10,070	4,355	4,313	2,33×

Todas as versões retornaram `64000012000000,0`, com erro zero em todos os níveis de otimização, incluindo `-O3`.

4.1 Ganho ao quebrar a dependência

Tomando a soma dependente como referência:

Acumuladores	-00	-02	-03
1	1,00×	1,00×	1,00×
2	1,95×	1,83×	1,87×
4	3,73×	2,81×	2,83×
8	4,82×	2,83×	2,88×

Duas variáveis quase dividiram o tempo por dois. Quatro produziram novo ganho. Com -02 e -03, passar de quatro para oito praticamente não alterou o tempo. Nesse ponto, mais acumuladores não escondem latência adicional: a largura do pipeline, as unidades de execução e a transferência de memória já estão próximas do limite útil para esse laço.

5. Efeito dos níveis de otimização

-00 mantém o código próximo da forma escrita e desativa a maior parte das transformações. Mesmo assim, separar os acumuladores melhorou o tempo, mostrando que o estilo do código já expõe instruções independentes ao processador.

-02 reduziu fortemente todos os tempos por meio de melhor uso de registradores, eliminação de trabalho de controle e vetorização automática quando permitida. Neste GCC, as opções de vetorização estão habilitadas tanto em -02 quanto em -03.

-03 não garantiu um programa mais rápido. Ele melhorou um pouco a inicialização, mas empatou com -02 nas somas; a soma dependente chegou a ficar ligeiramente mais lenta dentro da variação experimental. Otimização é uma tentativa do compilador, e seu resultado depende do laço e do processador.

6. Validação da vetorização

Foi feito um teste de controle com -02 -fno-tree-vectorize, desativando a vetorização automática:

Laço	-02 normal	-02 sem vetorização
Soma dependente	12,334 ms	12,677 ms
2 acumuladores	6,742 ms	6,686 ms
4 acumuladores	4,388 ms	5,039 ms
8 acumuladores	4,355 ms	5,202 ms

A soma dependente e a versão com dois acumuladores quase não mudaram. Com quatro e oito acumuladores, a vetorização trouxe cerca de 13% a 16% de ganho adicional. Portanto, a melhoria principal veio da quebra da dependência e do ILP; nas versões maiores, o compilador também aproveitou SIMD.

7. Conclusão

A dependência no único acumulador limita o paralelismo porque cada adição espera a anterior. Usar acumuladores independentes permite intercalar instruções e reduz as bolhas no pipeline. O ganho cresceu até quatro acumuladores e depois saturou nas compilações otimizadas. -02 e -03 foram muito superiores a -00, mas -03 não foi automaticamente melhor que -02. Os resultados mostram que desempenho depende tanto do código escrito quanto das transformações escolhidas pelo compilador.

8. Código-fonte

ilp.c · 241 linhas

```
1 /*
2  * Tarefa 3 - Paralelismo ao nível de instrução (ILP)
3  *
4  * Compilar:
5  * gcc -O0 -Wall -Wextra -std=c99 ilp.c -o ilp_00.exe
6  * gcc -O2 -Wall -Wextra -std=c99 ilp.c -o ilp_02.exe
7  * gcc -O3 -Wall -Wextra -std=c99 ilp.c -o ilp_03.exe
8  *
9  * Executar um teste por processo, evitando influencia entre as somas:
10 * ./ilp_02.exe inicializacao
11 * ./ilp_02.exe dependente
12 * ./ilp_02.exe multipla2
13 * ./ilp_02.exe multipla4
14 * ./ilp_02.exe multipla8
15 */
16
17 #define _POSIX_C_SOURCE 199309L
18
19 #include <stdio.h>
20 #include <stdlib.h>
21 #include <string.h>
22 #include <time.h>
23
24 #define TAMANHO 16000000
25 #define TEMPO_ALVO 0.6
26
27 /* Torna o resultado observável e impede a remoção dos testes pelo compilador. */
28 volatile double controle = 0.0;
29
30 double walltime(void)
31 {
32     struct timespec tempo;
33     clock_gettime(CLOCK_MONOTONIC, &tempo);
34     return tempo.tv_sec + tempo.tv_nsec / 1000000000.0;
35 }
36
37 /* Laco 1: inicializacao independente. Cada iteracao escreve uma posicao. */
38 __attribute__((noinline))
39 void inicializar(double *vetor, int n, double ajuste)
40 {
41     int i;
42
43     for (i = 0; i < n; i++) {
44         vetor[i] = i * 0.5 + 1.0 + ajuste;
45     }
46 }
47
48 /* Laco 2: uma unica soma cria uma cadeia de dependencias. */
49 __attribute__((noinline))
50 double soma_dependente(const double *vetor, int n)
51 {
52     double soma = 0.0;
53     int i;
54
55     for (i = 0; i < n; i++) {
56         soma += vetor[i];
57     }
58
59     return soma;
60 }
61
62 /* Laco 3a: duas cadeias de soma independentes. */
63 __attribute__((noinline))
64 double soma_multipla2(const double *vetor, int n)
65 {
66     double soma0 = 0.0;
67     double soma1 = 0.0;
68     int i;
69
70     for (i = 0; i + 1 < n; i += 2) {
71         soma0 += vetor[i];
72         soma1 += vetor[i + 1];
73     }
74     for (; i < n; i++) {
75         soma0 += vetor[i];
76     }
77
78     return soma0 + soma1;
79 }
80
81 /* Laco 3b: quatro cadeias de soma independentes. */
82 __attribute__((noinline))
```

```

83 double soma_multipla4(const double *vetor, int n)
84 {
85     double soma0 = 0.0, soma1 = 0.0;
86     double soma2 = 0.0, soma3 = 0.0;
87     int i;
88
89     for (i = 0; i + 3 < n; i += 4) {
90         soma0 += vetor[i];
91         soma1 += vetor[i + 1];
92         soma2 += vetor[i + 2];
93         soma3 += vetor[i + 3];
94     }
95     for (; i < n; i++) {
96         soma0 += vetor[i];
97     }
98
99     return soma0 + soma1 + soma2 + soma3;
100 }
101
102 /* Laco 3c: oito cadeias de soma independentes. */
103 __attribute__((noinline))
104 double soma_multipla8(const double *vetor, int n)
105 {
106     double soma0 = 0.0, soma1 = 0.0;
107     double soma2 = 0.0, soma3 = 0.0;
108     double soma4 = 0.0, soma5 = 0.0;
109     double soma6 = 0.0, soma7 = 0.0;
110     int i;
111
112     for (i = 0; i + 7 < n; i += 8) {
113         soma0 += vetor[i];
114         soma1 += vetor[i + 1];
115         soma2 += vetor[i + 2];
116         soma3 += vetor[i + 3];
117         soma4 += vetor[i + 4];
118         soma5 += vetor[i + 5];
119         soma6 += vetor[i + 6];
120         soma7 += vetor[i + 7];
121     }
122     for (; i < n; i++) {
123         soma0 += vetor[i];
124     }
125
126     return soma0 + soma1 + soma2 + soma3 +
127         soma4 + soma5 + soma6 + soma7;
128 }
129
130 double executar_soma(const char *teste, const double *vetor, int n)
131 {
132     if (strcmp(teste, "dependente") == 0) {
133         return soma_dependente(vetor, n);
134     }
135     if (strcmp(teste, "multipla2") == 0) {
136         return soma_multipla2(vetor, n);
137     }
138     if (strcmp(teste, "multipla4") == 0) {
139         return soma_multipla4(vetor, n);
140     }
141     return soma_multipla8(vetor, n);
142 }
143
144 void medir_inicializacao(double *vetor, int n)
145 {
146     int repeticoes = 1;
147     double tempo_total;
148     int r;
149
150     do {
151         double inicio = walltime();
152
153         for (r = 0; r < repeticoes; r++) {
154             inicializar(vetor, n, r * 0.001);
155             controle += vetor[r % n];
156         }
157
158         tempo_total = walltime() - inicio;
159         if (tempo_total < TEMPO_ALVO) {
160             repeticoes *= 2;
161         }
162     } while (tempo_total < TEMPO_ALVO);
163
164     printf("Teste: inicializacao\n");
165     printf("Elementos: %d\n", n);
166     printf("Repeticoes: %d\n", repeticoes);
167     printf("Tempo total: %.6f s\n", tempo_total);
168     printf("Tempo por laco: %.6f s\n", tempo_total / repeticoes);
169     printf("Validacao: primeiro=%.3f ultimo=%.3f\n",
170         vetor[0], vetor[n - 1]);
171 }

```

```

172
173 void medir_soma(const char *teste, double *vetor, int n)
174 {
175     int repeticoes = 1;
176     double tempo_total;
177     double resultado = 0.0;
178     double esperado = (double)n * (n - 1) / 4.0 + n;
179     int r;
180
181     inicializar(vetor, n, 0.0);
182
183     do {
184         double inicio = walltime();
185
186         for (r = 0; r < repeticoes; r++) {
187             resultado = executar_soma(teste, vetor, n);
188             controle += resultado;
189         }
190
191         tempo_total = walltime() - inicio;
192         if (tempo_total < TEMPO_ALVO) {
193             repeticoes *= 2;
194         }
195     } while (tempo_total < TEMPO_ALVO);
196
197     printf("Teste: %s\n", teste);
198     printf("Elementos: %d\n", n);
199     printf("Repeticoes: %d\n", repeticoes);
200     printf("Tempo total: %.6f s\n", tempo_total);
201     printf("Tempo por laço: %.6f s\n", tempo_total / repeticoes);
202     printf("Resultado: %.1f\n", resultado);
203     printf("Erro: %.1f\n", resultado - esperado);
204 }
205
206 int main(int argc, char *argv[])
207 {
208     double *vetor;
209     const char *teste;
210
211     if (argc != 2) {
212         printf("Uso: %s inicializacao|dependente|multipla2|multipla4|multipla8\n",
213             argv[0]);
214         return 1;
215     }
216
217     teste = argv[1];
218     if (strcmp(teste, "inicializacao") != 0 &&
219         strcmp(teste, "dependente") != 0 &&
220         strcmp(teste, "multipla2") != 0 &&
221         strcmp(teste, "multipla4") != 0 &&
222         strcmp(teste, "multipla8") != 0) {
223         printf("Teste invalido.\n");
224         return 1;
225     }
226
227     vetor = malloc((size_t)TAMANHO * sizeof(double));
228     if (vetor == NULL) {
229         printf("Nao foi possivel alocar o vetor.\n");
230         return 1;
231     }
232
233     if (strcmp(teste, "inicializacao") == 0) {
234         medir_inicializacao(vetor, TAMANHO);
235     } else {
236         medir_soma(teste, vetor, TAMANHO);
237     }
238
239     free(vetor);
240     return 0;
241 }

```